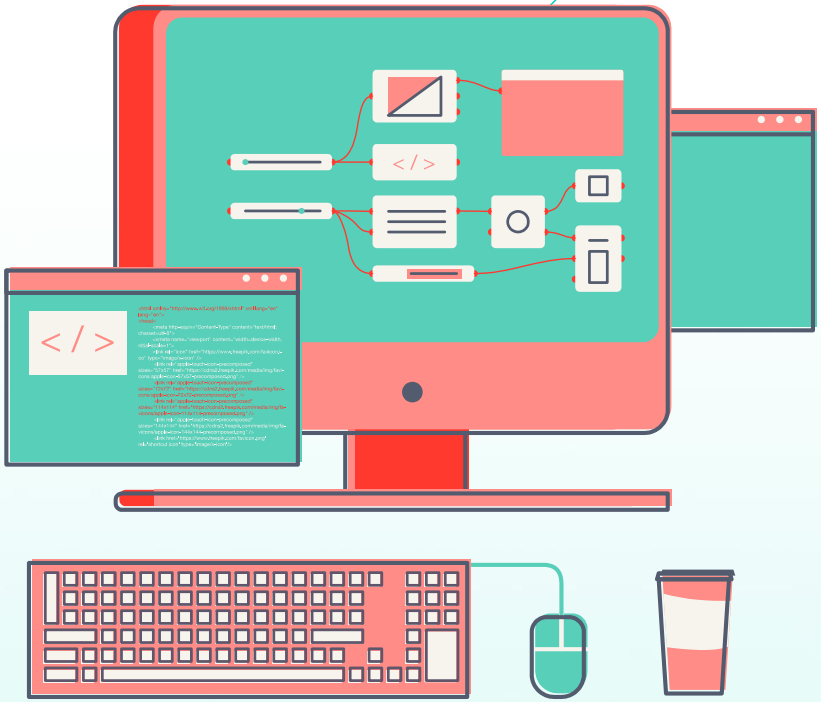
Computer Architecture Practice

Machine Level Programming 3

Marcell Fehér
sw0rdf1sh@ece.au.dk

Edited by Nhan Nguyen



Agenda

1. Processes
2. Fork
3. Fork example

Processes

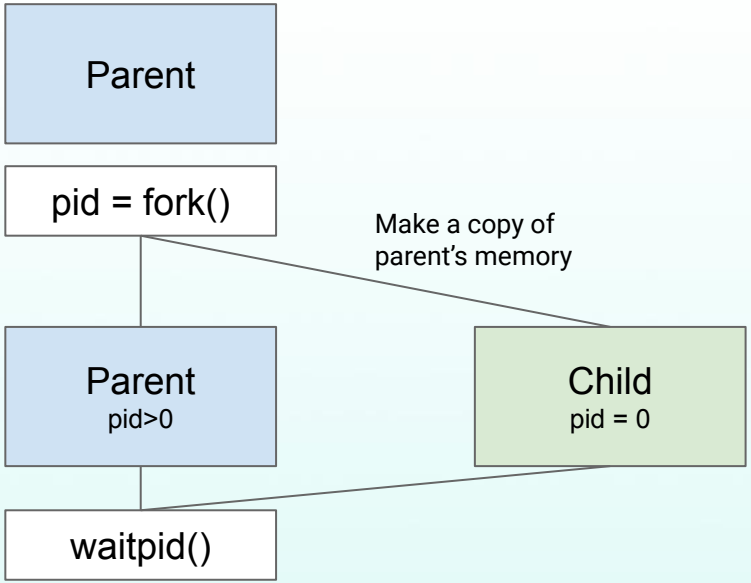
- Instance of a running program
- Fundamental abstraction in operating systems
- Each running application is a separate process (one or multiple)
- They cannot access each other’s memory (registers, stack and heap)
  - Good for the process: can use registers and allocate memory freely
  - Good for the OS: memory leak in one process doesn’t affect the others
  - Good for the user: security barrier around each application

Processes

- There are well-defined ways to communicate between processes (called InterProcess Communication, IPC)
- Some applications use multiple processes internally
  - Programs that run other programs, like the terminal/shell
  - Browsers run tabs and extensions in a separate process
  - Web servers use multiple processes to protect against memory leaks
  - Plugins are typically separate processes
  - Etc.

Fork

- Start a new process: **`fork()`** system call
- Wait for a child process to finish: **`waitpid()`** system call



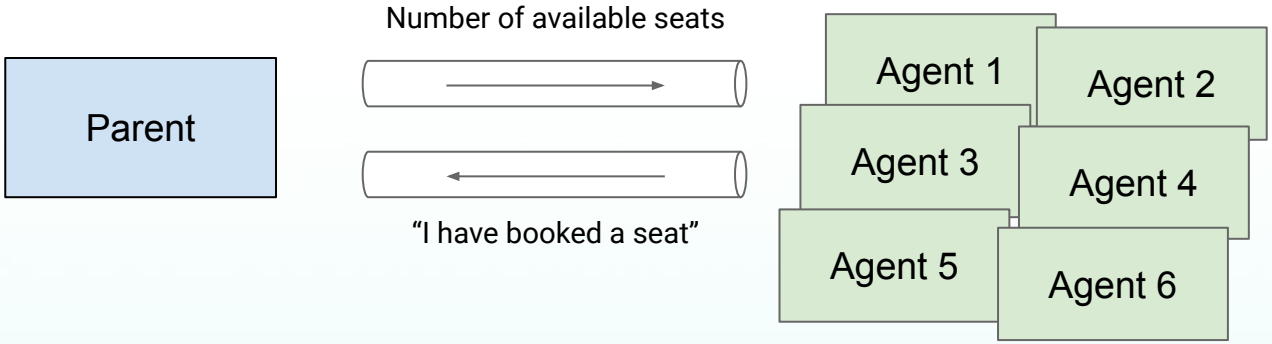
Fork example

- Looks simple on the flowchart, not that trivial in code
- We'll look at an example program
- "Travel agency" simulator:
  - We have a plane with a fixed number of seats
  - Seats are not differentiated
  - Processes act as travel agents selling tickets
  - They work in parallel, the parent makes sure there's no overbooking
  - Agent processes shut down when tickets are sold out
  - Parent waits for every child to finish before exiting

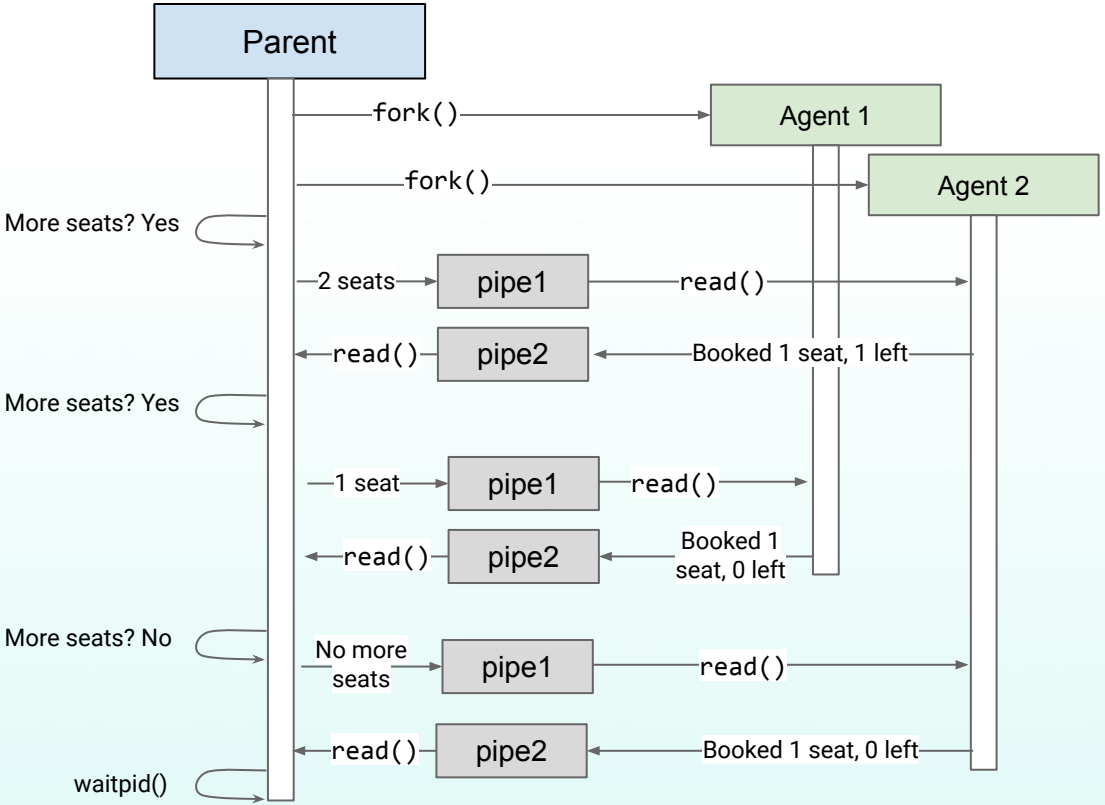
Fork example - IPC

- Parent (agency) and children (agents) need to communicate
  - Number of seats available
- We'll use the "pipe" mechanism for this
  - Very basic IPC, provided by the operating system
  - Works in Linux/macOS (under Windows you should use the Linux VM)
  - `write(pipe, message_ptr, message_length)`
  - `read(pipe, message_ptr, message_length)`
  - We'll use 2 pipes to avoid sync issues: parent → child, child → parent
  - Message can be received only 1x
    - Even if multiple agents are listening, only 1 will receive the message

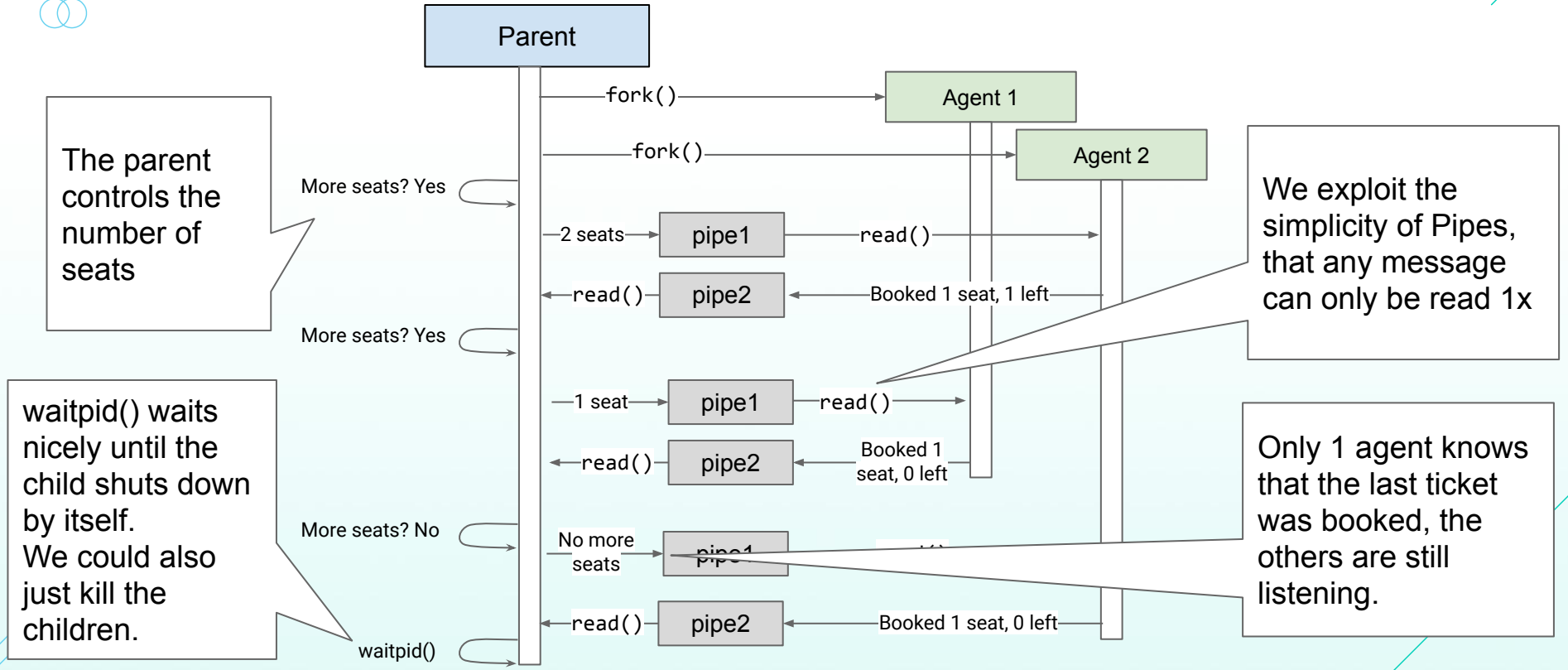
Fork example - IPC



Fork example - IPC



Fork example - IPC



Walk through the code together with the TA!

Blackboard: Exercises/Week 6 (MLP3)/fork_example.cpp

Compile: *g++ fork_example.cpp -o fork.out*

Run: *./fork.out*

Does not work in Windows, use the VM!

Questions

- **Why do we need to collect the children process IDs?**

Questions

- **Do the agents know the process IDs of other agents?**

Questions

- **Why is it safe to declare the `seats_left` variable in the agent function?** (e.g. why isn't this messing up the other agents?)

Questions

- **Explain what is the meaning of the `seats_left` variable when it's positive, zero and negative!**

Questions

- **What guarantees that when no more seats are available, each agent gets the information 1x?** (e.g. why is it safe to decrease `seats_left` in the agent function when there are not more seats?)

Questions

Put the following events in the right order!

- All agent processes have exited
- Parent exits
- All tickets are sold
- Last waitpid() call returns
- Parent calls waitpid()
- The first agent process exits

Homework

Make 2 copies of the code, and change process management in the following ways:

- The child process exits immediately when it notices there's no more available seats
- The parent kills the children processes instead of waiting for them to finish

Both strategies require less code than the original version. Think about what coordination is not needed, and remove all the unnecessary code.